

## Stage 5 concrete outlines: domain-embedded TanStack Trust

### Status and boundary

This is the outline pass of the Editorial Design Assay for TT-RFC-BAC. Kyle selected the revised F3 family: use cases supply the forward motion while domain work, base mechanics, and failure/repair remain visible together. The base/domain seam is load-bearing. TanStack Trust does not manufacture trust; it supplies reusable machinery with which a domain defines, inspects, maintains, and repairs its own grounds for reliance.

The three outlines below are unranked. They vary section sequence and function, not the theory or source boundary. They were generated in one informed context and are therefore correlated. None is an approved outline, title, or draft.

### Frozen presentation baseline

Software packages need a shared way to define domain-specific claims and checks, record execution-linked evidence, assess explicit support routes under current conditions, retain contradiction and repair history, and expose that state to humans and agents without deciding whether a consumer may act. The reusable base can own these mechanics, transport, and status semantics. A domain package must own the meaning of its claims, the authority of its rules and checks, the adequacy of evidence, and applicability. Consumer policy owns permission and fallback.

The current implementation is a bounded local prototype. TanStack DB Endpoints is one implemented domain consumer and not evidence that the architecture generalizes. Generic package authoring, richer applicability, rule and checker soundness, agent-workflow effectiveness, installed clients, independent range, and production operation remain open.

### Frozen family and repeated reading controls

Every outline uses the same three-part braid at each substantive use case:

1. **Domain work:** what a person or agent is trying to claim, inspect, or change, and what the domain package must know.
2. **Base mechanics and authority:** what the base records, computes, exposes, or refuses to infer, including the owner of every semantic decision.
3. **Failure and repair:** the concrete witness that justifies the boundary and what remains unresolved.

Every section also carries a compact status band distinguishing **current implementation**, **bounded example**, **normative guidance**, **proposed contract**, **absent operation**, and **unmeasured outcome**. Evidence state, execution/reach state, and consumer policy remain separate; they are never compressed into one trust score.

The outlines target a 5,500–6,500 word RFC for Kyle. Long source provenance and loss ledgers remain linked support. The RFC itself must contain the authority map, state-axis table, CLI/LSP/MCP capability matrix, borrowed-mechanisms ledger, and unresolved-decision register.

## Outline O1 — One change through the evidence lifecycle

### Organizing rule

Follow one hypothetical agent-produced software change forward from proposed claim to consumer decision. The change is generic; an Endpoints safe-skip example runs beside it as a bounded illustration. Each step advances time and shows domain ownership, base behavior, and the failure that constrains it.

### Intended reader movement

Kyle begins with the cheap-code/trust problem and ends able to replay how a domain-owned guarantee gains, loses, and may regain current support. The architecture appears when the journey first crosses an authority boundary.

### Detailed structure

#### 1. Coding is cheap; reliance is not — 450–550 words

- **Question:** What remains scarce when agents can generate a change quickly?
- **Source material:** Kyle’s product framing; AIUC’s capability-versus- deployability analysis as adjacent problem language; Stage 4 reader promise.
- **Necessary mechanism/example:** Contrast code production with the unanswered questions: what is claimed, what supports it, where does it apply, and who decides. State immediately that this is not agent certification or a generic trust product.
- **Transition:** If trust is local to a domain, the first task is to name the domain claim rather than score the agent.
- **Disclosure:** AIUC is an interested founder account and supports framing, not the architecture or its effectiveness.
- **Contribution:** Establishes the reader’s reason to care without promising that TanStack Trust makes agents trustworthy.

#### 2. The domain defines what the change must preserve — 650–750 words

- **Question:** What exactly is being claimed, and who has authority to define it?
- **Source material:** ESS-S01, ESS-S02, ESS-S06, ESS-S08, ESS-T03; Design Grammar primitives; design-check workflow.
- **Necessary mechanism/example:** A claim package owns the law, bounded domain, rule, checker, relevant conditions, omissions, and replay obligation. The base owns none of those meanings. Introduce the generic claim beside endpoints/safe-skip-refetch@1, explicitly labeled a bounded example and not range evidence.
- **Transition:** Once the domain has declared the promise and its test, the system still needs evidence that the test actually ran and reached what it claims to observe.
- **Disclosure:** Generic public package authoring and rule registration are proposed or absent; registered rule and checker soundness remain trust roots.
- **Contribution:** Makes the base/domain seam concrete before the base is allowed to compute anything.

### 3. A run becomes evidence only through a declared contract — 700–850 words

- **Question:** What turns a test result into admissible evidence for this claim?
- **Source material:** ESS-S04–S06, ESS-H04, ESS-P05, ESS-E04–E07; Oracle Guide and CheckContract.
- **Necessary mechanism/example:** Separate run provenance, per-law observations, reach, omissions, fault controls, fixture responsibility, and replay. Use AIUC’s controls-versus-effectiveness distinction only as a framing echo. Show that exceptions and non-reach do not enter evidence history as correctness observations.
- **Transition:** Admissible observations still do not answer whether the claim is currently supported; the base must evaluate the declared argument.
- **Disclosure:** Failure receipts, mutation outcomes, and complete environment capture are narrower than the Oracle Guide; workflow effectiveness is unmeasured.
- **Contribution:** Replaces “the tests passed” with an inspectable evidentiary contract.

### 4. The base assesses support without creating meaning — 700–800 words

- **Question:** Given the domain’s rules and observations, what can the base legitimately conclude?
- **Source material:** ESS-S02–S05, ESS-H02, ESS-T04; AND/OR repair and cycle tests.
- **Necessary mechanism/example:** Walk an argument whose premises are AND within a route and OR across routes; contrast recursive support with a flat gap inventory. Show no evidence, unresolved evidence, contradiction, operational non-reach, and consumer denial as different states. Display the evidence/reach/policy table here.
- **Transition:** Even a correctly assessed route may cease to apply when its dependencies or requested context change.
- **Disclosure:** The prototype implements a finite local model; it does not prove arbitrary rules or define universal missing-evidence severity.
- **Contribution:** Gives Kyle the reusable kernel’s positive contract and its most important refusal.

### 5. Current support depends on current context — 600–750 words

- **Question:** Why is an old green result not automatically evidence now?
- **Source material:** ESS-S04, ESS-H07, ESS-P01, ESS-P03, ESS-T01, ESS-T02, ESS-T05.
- **Necessary mechanism/example:** Fingerprint  $a \rightarrow b \rightarrow a$  does not resurrect the old observation. Put the grammar’s rule-owned three-valued applicability relation beside the kernel’s private boolean exact matcher. Keep ownership and cardinality as separate unresolved choices.
- **Transition:** Context withdrawal can make support stale; a direct failing observation creates a different problem—a retained contradiction.
- **Disclosure:** Complete dependency capture, semantic equivalence, subsumption, and cross-version mapping remain unsolved.
- **Contribution:** Makes freshness and applicability part of meaning rather than metadata.

### 6. Contradiction survives unrelated success — 550–650 words

- **Question:** What happens when the domain observes a counterexample?
- **Source material:** ESS-S04, ESS-S05, ESS-H03, ESS-P02, ESS-P05; Field Log repair witnesses.

- **Necessary mechanism/example:** A failing observation opens a challenge. Unrelated green evidence, a late-delivered run that began before the failure, and a later item in the same returned batch cannot erase it.
- **Transition:** Preserving the contradiction creates the next task: establish whether a new execution truly revisited the same case.
- **Disclosure:** These are reproduced local model failures, not production Endpoints incidents.
- **Contribution:** Explains why append-oriented history is necessary for trustworthy maintenance.

#### 7. Repair appends a new reason; it does not rewrite history — 650–750 words

- **Question:** Under what conditions may current support be restored?
- **Source material:** ESS-S04, ESS-S08, ESS-H03, ESS-P02/P03, ESS-T08; repair-rule workflow and v2 rules.
- **Necessary mechanism/example:** A causally later exact replay may append a resolution; the original challenge and resolution remain. A later context change can expire the resolution's current authority. Separate base-checked invocation causality from the trusted checker's measurement freshness.
- **Transition:** The lifecycle is now inspectable; the practical question is which parts humans and agents can actually operate through today's public interfaces.
- **Disclosure:** Alternate strategy repair, selective expiry, durable repair certificates, and distributed clocks remain open.
- **Contribution:** Shows repair as evidentiary work rather than status mutation.

#### 8. Agents can inspect part of the system today — 650–800 words

- **Question:** What can a coding agent or editor actually ask or do?
- **Source material:** ESS-S09, ESS-P06/P07, ESS-E10; grammar-to-interface loss ledger.
- **Necessary mechanism/example:** Present the semantic-capability matrix for CLI, LSP, and MCP over one service: status, assessment, history, context, resolution, and the Endpoints check where implemented. Mark generic package authoring, rule registration, argument proposal, and generic evidence ingestion as absent. Note that LSP diagnostic severity is adapter policy.
- **Transition:** Once evidence is visible through tools, responsibility passes to the consumer rather than back into the base.
- **Disclosure:** Shared transport parity is one implementation path, not independent corroboration; installation and real-client agent use are unfinished.
- **Contribution:** Lets Kyle distinguish an agent-operable concept from the narrower public surface that exists.

#### 9. The consumer decides—and the base stays small — 550–700 words

- **Question:** Who finally decides whether the change may be merged, shipped, or used?
- **Source material:** ESS-S01, ESS-S05, ESS-H01/H05/H06, ESS-T04/T09; Stage 4 hostile assay; source-transfer readings.
- **Necessary mechanism/example:** Consumer policy sets severity, permission, and fallback. Close with the architecture/status reference plane, the borrowed-mechanisms-and-breakpoints

ledger (Lean, Field Lab, shadcn/lint, Beads, AIUC, Endpoints), and the unresolved-decision register.

- **Transition:** Final synthesis returns to the opening question: the base is valuable only insofar as it helps a domain maintain better grounds for reliance.
- **Disclosure:** No generality, adoption, workflow-effectiveness, production, security, or outcome-improvement claim is made.
- **Contribution:** Prevents the lifecycle story from ending in a universal trust verdict.

### Principal losses and risks

- The forward journey can make the prototype’s exact replay model look like the inevitable general lifecycle.
  - Endpoints appears throughout and may quietly become the generic claim despite repeated boundaries.
  - Independent lookup by architecture owner or interface capability is slower; the reference tables must carry that burden.
  - Applicability and consumer policy arrive late enough that an impatient reader may initially overestimate the base.
  - Chronology can turn “this rule followed a reproduced failure” into a stronger causal or general validity claim than the sources support.
- 

## Outline O2 — Three owners and their handoffs

### Organizing rule

Organize the RFC around responsibility handoffs rather than one changing claim: the domain package defines meaning, the base maintains evidence mechanics, and the consumer decides action. Within each owner section, use cases braid normal work with the failure and repair obligations that owner cannot delegate.

### Intended reader movement

Kyle first receives the product boundary, then inspects each owner and the contracts between them. The Endpoints walkthrough arrives only after the abstract handoffs are clear and is used to test whether the ownership map can be reconstructed in one bounded case.

### Detailed structure

#### 1. The product opportunity is a coordination problem — 450–550 words

- **Question:** Why does cheap code increase the need for explicit ownership?
- **Source material:** Kyle’s framing; AIUC problem analysis with non-transfer; ESS-S01; Stage 4 reader promise.
- **Necessary mechanism/example:** More generated changes increase the number of claims, checks, reviews, and later repairs someone must coordinate. Trust remains domain-local; the reusable opportunity is common evidence machinery.
- **Transition:** Name the three decision owners before describing any component.

- **Disclosure:** This is a product rationale, not measured developer behavior.
- **Contribution:** Frames TanStack Trust as shared responsibility machinery, not a trust authority.

## 2. The authority map: domain, base, workflow, interface, consumer — 700–850 words

- **Question:** Which layer may decide what?
- **Source material:** ESS-S01, ESS-S02, ESS-S05, ESS-S09, ESS-H01; Stage 4 hostile constraints.
- **Necessary mechanism/example:** Present one orthogonal reference plane: ownership layers, six semantic primitives, the evidence/reach/policy axes, and the status legend. Show the base as a small mechanism surrounded by domain-owned semantics and consumer-owned permission.
- **Transition:** The first handoff begins when a domain author turns a useful software guarantee into an executable evidence contract.
- **Disclosure:** This is a proposed general architecture with only a bounded partial implementation.
- **Contribution:** Gives the rest of the RFC a stable coordinate system and blocks overarching-product drift early.

## 3. The domain package owns the promise — 750–900 words

- **Question:** What must a domain encode before the base can help?
- **Source material:** ESS-S02, ESS-S06–S08, ESS-H04, ESS-T01/T03/T05/T07; design-check workflow and Oracle Guide.
- **Necessary mechanism/example:** Claims, laws, bounded domain, checks, applicability, omissions, fault controls, and replay obligations. Contrast control presence with demonstrated effectiveness. Introduce deterministic, empirical/oracle, and rubric-assessed forms without ranking them.
- **Transition:** Once the domain supplies meaning and an admissible route, the base can record execution without pretending to validate the checker itself.
- **Disclosure:** Applicability ownership/cardinality, checker soundness, rubric admission, and complete dependency discovery remain open.
- **Contribution:** Makes the domain layer substantial enough that the base cannot be mistaken for the source of trust.

## 4. The base owns evidence mechanics, not domain truth — 850–1,000 words

- **Question:** What reusable work can the base perform safely?
- **Source material:** ESS-S02–S05, ESS-S11, ESS-P01–P05/P08, ESS-E08/E09/E11/E12.
- **Necessary mechanism/example:** Record runs and observations; assess AND/OR routes; exclude cycles; track explicit context; retain challenges; require later exact replay; import/export a checksummed local state. Pair each mechanism with its limiting witness: flat gaps,  $a \rightarrow b \rightarrow a$ , late or same-batch replay, expired resolution, reach failure, and non-hostile checksum.
- **Transition:** These mechanics become useful only when workflows and public interfaces let people and agents operate them correctly.
- **Disclosure:** Exact identity, global epochs, local serialization, atomic rename, and corruption detection are conservative prototype boundaries—not universal semantics, multi-process durability, or security.
- **Contribution:** Defines the small reusable core through both powers and refusals.

## 5. Workflows carry maintenance obligations across the handoff — 600–750 words

- **Question:** How do authors and agents keep evidence meaningful after the first passing run?
- **Source material:** ESS-S08, ESS-H04/H05, ESS-E03–E07; three workflow cards.
- **Necessary mechanism/example:** Design, establish/maintain, and repair are separate procedures. Show counterexample retention, reach and fault controls, failure provenance, same-violation replay, and matched correction evaluation as proposed future work.
- **Transition:** The procedures need an operable surface; documentation alone does not make the generic system available to agents.
- **Disclosure:** The workflows are documented but untested with coding agents; contextual diagnostics and correction evaluation remain unimplemented.
- **Contribution:** Makes “maintainable evidence” a recurring job rather than a kernel side effect.

## 6. Interfaces expose one service unevenly — 650–800 words

- **Question:** Which responsibilities are reachable through CLI, LSP, and MCP?
- **Source material:** ESS-S09, ESS-P06/P07, ESS-E10; public-interface loss audit.
- **Necessary mechanism/example:** Use a semantic-capability matrix rather than protocol screenshots. Mark status, assess, history, context, resolve, and the Endpoints-specific check where present; mark generic authoring, registration, argument proposal, and ingestion absent. Separate LSP severity from base status.
- **Transition:** The matrix makes the remaining handoff visible: tools can report support, but they cannot own the consumer’s decision.
- **Disclosure:** One shared service establishes path consistency only; real editor and agent integration is unmeasured.
- **Contribution:** Turns “agents can use it” into an exact, falsifiable surface claim.

## 7. Consumer policy owns permission and fallback — 500–650 words

- **Question:** What does a supported, contradicted, unresolved, or unreachable result permit?
- **Source material:** ESS-S05, ESS-H06, ESS-T04/T09; Endpoints design boundary.
- **Necessary mechanism/example:** Compare evidence state, reach state, and policy decision. Show why missing evidence need not universally veto an optimization and why “full refetch” is not always a safe generic fallback.
- **Transition:** With the owners separated, run one complete Endpoints case through the map to see where the implementation actually reaches.
- **Disclosure:** No universal strictness, severity, priority, or fallback is selected.
- **Contribution:** Completes the authority chain without smuggling product policy into the base.

## 8. Endpoints: one bounded reconstruction of the handoffs — 700–850 words

- **Question:** Can the ownership model explain a real domain package without turning that package into the architecture?
- **Source material:** ESS-S10, ESS-P04, ESS-E02/E07/E13; Endpoints check card and extracted decision helper.
- **Necessary mechanism/example:** Walk disjoint complete bounds (pass), overlap (fail), and incomplete effects (unresolved). Identify which facts the fixture supplies, what canSkipRefetch

computes, what the base records, what the interfaces expose, and what consumer enforcement still owns.

- **Transition:** The case demonstrates the seam; the final section asks where the model came from and what remains unproved.
- **Disclosure:** Pre-analyzed summaries, SQL analyzer completeness, PostgreSQL range, external writes, production policy, and generality remain outside.
- **Contribution:** Gives the architecture a concrete audit without using the example as independent range evidence.

## 9. Borrowed mechanisms, breakpoints, and the decision register — 600–750 words

- **Question:** Which parts were adapted, where do they stop, and what must Kyle still decide?
- **Source material:** Stage 4 source-transfer results; AIUC transfer and user correction; ESS-H01/H05; Stage 2 theoretical/evidence gaps.
- **Necessary mechanism/example:** Compact ledger for Lean, Field Lab, shadcn/lint, Beads, AIUC, and Endpoints. For each: mechanism, target use, evidence status, nearby break, and prohibited inference. Follow with the unresolved theoretical, evidence, implementation, and product decisions.
- **Transition:** Close on the evaluation standard: future work must improve a domain's ability to maintain evidence, not merely add another adapter or component.
- **Disclosure:** Integration for Kyle is the supported contribution; novelty and general effectiveness are not established.
- **Contribution:** Leaves Kyle with an ownership-correct roadmap rather than a polished fiction of completion.

### Principal losses and risks

- Leading with the authority map can feel like an architecture document before the product need has accumulated enough force.
- Grouping by owner fragments the causal sequence by which one observation becomes support and later repair.
- The concentrated Endpoints section may carry too much explanatory weight even though it appears late.
- Workflow and interface sections can read as component inventory unless every passage stays attached to a concrete handoff.
- The base may look cleaner and more settled than it is because unresolved semantics are gathered at layer boundaries.

---

## Outline O3 — The decision-backward trust review

### Organizing rule

Begin at the moment a human or agent wants to rely on a generated change. Work backward through the questions required to justify that reliance, then forward through contradiction and

repair. Each question exposes the responsible domain semantics, the base’s bounded answer, and the failure that prevents a shortcut.

### **Intended reader movement**

Kyle starts with the user-visible decision rather than system components. By reconstructing the grounds for that decision, he discovers why the base/domain seam, evidence lifecycle, and missing public operations exist. The final sections convert the review into an evaluable product boundary and work list.

### **Detailed structure**

#### **1. An agent proposes a change. May anything rely on it? — 500–650 words**

- **Question:** What would a useful answer to “can we trust this?” actually contain?
- **Source material:** Kyle’s cheap-code framing; AIUC bounded-promise analysis with architectural non-transfer; ESS-S05; reader-promise result.
- **Necessary mechanism/example:** Replace a scalar trust question with five review questions: what claim, what evidence, where applicable, what defeats it, and who decides. Preview the three state axes and status legend.
- **Transition:** The first review question is semantic, so it cannot be answered by the base or by the agent’s confidence.
- **Disclosure:** No measurement shows that agent code volume has already made this system effective or necessary for TanStack users.
- **Contribution:** Gives the RFC an immediate use case and a non-overarching definition of trust.

#### **2. What domain promise would this change have to preserve? — 650–800 words**

- **Question:** Which claim is the reviewer actually evaluating?
- **Source material:** ESS-S01/S02/S06/S08, ESS-T03/T05; claim-package and workflow material.
- **Necessary mechanism/example:** Domain law, scope, rule, checker, relevant context, omissions, and replay obligation. Use Endpoints’ safe-skip claim as one bounded answer while retaining the generic question.
- **Transition:** A well-formed promise names what evidence would count but does not establish that the evidence exists.
- **Disclosure:** Generic package authoring is absent; rule/checker soundness and complete dependency capture remain domain trust roots.
- **Contribution:** Locates trust inside a domain commitment rather than an overarching platform.

#### **3. What evidence was actually produced? — 650–800 words**

- **Question:** Did the declared check execute, reach the production path, and produce observations admissible to the rule?
- **Source material:** ESS-S04–S06, ESS-H04, ESS-P05, ESS-E04–E07; Oracle Guide.
- **Necessary mechanism/example:** Run provenance, observations, reach, omissions, reference and production paths, fault controls, original/reduced traces, and fixture responsibility. Contrast configuration evidence with effectiveness evidence.
- **Transition:** Even real observations may fail to complete the argument for the claim.

- **Disclosure:** Current receipts and failure provenance are incomplete relative to the guide.
- **Contribution:** Makes “evidence-backed” inspectable rather than rhetorical.

#### 4. Does that evidence complete a valid support route? — 650–750 words

- **Question:** Which required premises are satisfied, and which alternative routes remain?
- **Source material:** ESS-S02–S05, ESS-H02; AND/OR and cycle repairs.
- **Necessary mechanism/example:** Recursive argument assessment, joint premises, alternative routes, cycle exclusion, and flat-gap projection. Show evidence status separately from operational reach and consumer policy.
- **Transition:** A complete route answers only for the context in which its evidence remains applicable.
- **Disclosure:** Finite prototype behavior is not proof of general logical or semantic completeness.
- **Contribution:** Shows the exact reusable inference that the base may own.

#### 5. Is the support current for this request? — 600–750 words

- **Question:** Has code, data, method, configuration, environment, or requested context changed?
- **Source material:** ESS-S04, ESS-P01/P03, ESS-T01/T02/T05, ESS-H07.
- **Necessary mechanism/example:**  $a \rightarrow b \rightarrow a$ ; applicability divergence between grammar and kernel; expiring current resolution authority. Put the unresolved ownership and cardinality decision directly beside the current exact matcher.
- **Transition:** If current evidence contradicts the claim, staleness is no longer the right description; the review must preserve a challenge.
- **Disclosure:** Semantic equivalence, selective invalidation, and complete dependency discovery are open.
- **Contribution:** Prevents old success from impersonating current support.

#### 6. What evidence defeats the claim—and can it be erased? — 550–700 words

- **Question:** How does the system treat a relevant failing observation?
- **Source material:** ESS-S04/S05, ESS-H03, ESS-P02/P05; repair history.
- **Necessary mechanism/example:** Retained challenge, unrelated success, late-delivered pass, and same-batch replay. Explain why provenance order alone is insufficient without a causality proxy.
- **Transition:** The only legitimate next question is whether new evidence revisited the same case after the contradiction existed.
- **Disclosure:** The witnesses are local constructed failures and retain exact identity assumptions.
- **Contribution:** Makes counterevidence first-class and prevents a status registry interpretation.

#### 7. What would count as repair? — 600–750 words

- **Question:** Which later work restores current authority without deleting the failure?
- **Source material:** ESS-S04/S08, ESS-P02/P03, ESS-T08; v2 repair and repair workflow.
- **Necessary mechanism/example:** Exact causally later replay, retained resolution history, expiring applicability, and separate checker-freshness trust. Name alternate repair strategies as unselected.

- **Transition:** With the evidentiary answer reconstructed, ask whether agents can retrieve or change any of it through current product surfaces.
- **Disclosure:** Distributed timing, alternate strategy repair, and durable repair certificates are absent.
- **Contribution:** Shows how the system earns a new bounded reason rather than merely flipping red to green.

#### 8. Can an agent perform this review through the product? — 650–800 words

- **Question:** Which review questions have public CLI, LSP, or MCP operations?
- **Source material:** ESS-S09, ESS-P06/P07, ESS-E10; interface loss ledger.
- **Necessary mechanism/example:** Semantic-capability matrix with present and absent operations. Distinguish shared-service consistency, LSP diagnostic policy, and MCP structured results. Keep Endpoints-specific execution visibly separate from generic authoring and ingestion.
- **Transition:** Tool access can expose evidence but still cannot decide the user’s acceptable risk or supply missing domain authority.
- **Disclosure:** No installed-client, real-agent workflow, correction-quality, or distribution evidence exists.
- **Contribution:** Tests the product vision against actual agent operability.

#### 9. Who owns the final decision, and what kind of product is this? — 650–800 words

- **Question:** After the review, what does TanStack Trust own—and refuse to own?
- **Source material:** ESS-S01/S05, ESS-H01/H05/H06, ESS-T04/T09; hostile and AIUC source-transfer results.
- **Necessary mechanism/example:** Reveal the complete ownership/reference plane assembled by the prior questions. Consumer policy decides action; domain packages define trust semantics; the base maintains evidence mechanics. Include the borrowed-mechanisms-and-breakpoints ledger here.
- **Transition:** The final section tests the product against everything still missing rather than closing with a universal claim.
- **Disclosure:** AIUC’s certification/insurance shape, Lean’s proof authority, shadcn/lint correction claims, Beads’ backend, and Endpoints generality do not transfer.
- **Contribution:** Answers the opening decision while preserving the local, layered nature of trust.

#### 10. What remains untrusted, unimplemented, or undecided? — 500–650 words

- **Question:** What would have to be designed or observed before this becomes a general agent-usable base?
- **Source material:** ESS-T01–T09, ESS-E01–E13; Stage 4 hostile assay and status requirements.
- **Necessary mechanism/example:** Group the register into semantics, authoring/API, validation/range, operations/security, and outcome evidence. Separate decisions from implementation tasks and empirical gaps.
- **Transition:** Close with evaluation criteria for the next prototype rather than a feature roadmap silently invented by the RFC.

- **Disclosure:** The RFC may clarify the decisions but cannot resolve them.
- **Contribution:** Leaves Kyle able to evaluate the current system and choose later work with no hidden architecture decisions.

### Principal losses and risks

- Working backward can make authoring and maintenance feel subordinate to inspection even though the domain must design the claim first in practice.
- The repeated questions may read like a checklist and hide the exploratory design history that produced the model.
- Revealing the full architecture late risks temporary confusion about which layer owns each earlier answer; the status and owner band must remain visible from the start.
- Starting at a merge/use decision may imply that consumer policy is always binary or that code review is the only intended setting.
- The strong opening question “can we trust this?” can still invite a scalar answer unless every section preserves claim, evidence, applicability, and policy separately.

---

### Cross-outline comparison to the frozen baseline

| Dimension                  | O1: lifecycle                                               | O2: ownership handoffs                                          | O3: decision-backward review                             |
|----------------------------|-------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------|
| Primary forward motion     | Time: claim → evidence → freshness → repair → use           | Responsibility: domain → base → workflow/interface → consumer   | Questions required to justify one reliance decision      |
| Base/domain seam           | Introduced at claim design and repeated through the journey | Established immediately and used as the whole section structure | Reconstructed one question at a time, then named in full |
| Failure/repair function    | Causal turning point in the lifecycle                       | Constraint on each owner’s delegated responsibility             | Adversarial test of each proposed shortcut               |
| End-points role            | Bounded sidecar across sections                             | One concentrated reconstruction after the abstract model        | Recurring answer to the review questions                 |
| Interface matrix           | Near the end of the lifecycle                               | Handoff between workflow and consumer                           | Test of whether the review is agent-operable             |
| Borrowed-mechanisms ledger | Closing architecture/policy section                         | Closing decision register                                       | Product-boundary synthesis before open gaps              |

| Dimension              | O1: lifecycle                             | O2: ownership handoffs             | O3: decision-backward review                        |
|------------------------|-------------------------------------------|------------------------------------|-----------------------------------------------------|
| Strongest preservation | Lifecycle causality and repair logic      | Authority and product architecture | Reader use case and bounded-promise logic           |
| Most exposed loss      | Lookup and early applicability visibility | Causal continuity                  | Authoring chronology and early architecture clarity |

All three preserve the candidate’s claim, confidence, causal limits, source roles, and unresolved range. O1 makes time newly salient; O2 makes ownership newly salient; O3 makes the consumer’s questions newly salient. None supplies new evidence that the base generalizes or improves agent output.

### Shared misfit register

No outline may absorb the following into a solved narrative:

- who owns applicability and whether it returns true/false or true/false/unknown;
- semantic equivalence, subsumption, or cross-version counterexample mapping;
- rule, checker, analyzer, oracle, or dependency-capture soundness;
- generic package authoring, rule registration, argument proposal, and evidence ingestion through public agent interfaces;
- speculative challenge objects, rubric admission, alternate repair, selective expiry, and durable repair certificates;
- missing-evidence severity, permission, confidence aggregation, investigation priority, and consumer fallback;
- independent range beyond the Endpoints-shaped lineage;
- demonstrated human or coding-agent workflow effectiveness;
- complete failure receipts, mutation receipts, installed clients, distribution, multi-process durability, hostile-process integrity, remote authority, scale, and production readiness; or
- novelty, market adoption, or measured improvement in generated software.

### Outline-choice checkpoint

The next step is one exact choice: select O1, O2, or O3; combine named sections while retaining one dominant organizing rule; request a bounded revision; reject all three; or stop. No title, subtitle, description, public packaging, blind outline probe, or prose draft begins before that choice.